



Iniciação Científica & Tecnológica



Universidade Presbiteriana
Mackenzie

AUTOMAÇÃO DE TRANSCRIÇÃO E ANÁLISE DE REUNIÕES UTILIZANDO INTELIGÊNCIA ARTIFICIAL

Matheus Barbosa Meloni — aluno pesquisador

Prof. Victor Inácio de Oliveira — orientador

Escola de Engenharia | Engenharia da Computação | Campus Higienópolis

Programa de Iniciação Científica (PIBIC/PIVIC) 2025–2026

Apoio institucional: Universidade Presbiteriana Mackenzie / CNPq

INTRODUÇÃO

Problema

Reuniões concentram decisões, pendências e encaminhamentos, mas o registro depende de anotação manual: é custoso, parcial e raramente rastreável ao que foi efetivamente dito.

Contexto tecnológico

- Reconhecimento automático de fala (ASR) atingiu robustez multilíngue com modelos tipo Whisper.
- Embeddings de sentença (Sentence-BERT) permitem medir proximidade semântica entre trechos.
- Modelos seq2seq (T5) permitem condensar texto sob instrução em linguagem natural.

Lacuna identificada

As soluções comerciais equivalentes são proprietárias, orientadas ao inglês e dependentes de nuvem — o que impede auditoria do processo e levanta restrições de privacidade sobre o áudio.

Objetivo geral

Desenvolver um sistema modular, reproduzível e executável localmente que converta áudio de reunião em português em documentação acionável: decisões, pendências e próximos passos.

Objetivos específicos

- Integrar ASR, estruturação semântica e sumarização num pipeline único e instrumentado.
- Isolar o núcleo de IA das camadas de uso, tornando-o reutilizável e testável.
- Definir um protocolo de avaliação reproduzível para medir desempenho e qualidade.

Arquitetura do pipeline — cinco estágios encadeados

Abordagem de pesquisa aplicada e experimental: o problema é decomposto em etapas com responsabilidade única e saída bem definida para a etapa seguinte.

1. Pré-processamento do áudio (librosa) — remoção de silêncio nas extremidades ($\text{top_db} = 25$), normalização de pico, subtração espectral em blocos de 60 s e reamostragem para 16 kHz mono.
2. Transcrição / ASR (faster-whisper) — decodificação por feixe ($\text{beam_size} = 5$), filtro de atividade de voz (VAD) e idioma fixado em pt; saída segmentada com marcação temporal.
3. Estruturação semântica (Sentence-BERT) — tokenização e reorganização em janelas deslizantes de 500 tokens com sobreposição de 50; cada janela vira um vetor normalizado em L2.
4. Agrupamento temático — similaridade de cosseno contra limiar de 0,78, com fusão incremental gulosa e atualização do centroide; cada bloco é remapeado ao eixo temporal da reunião.
5. Sumarização (T5) — instruções em linguagem natural por seção (decisões / pendências / próximos passos), com corte controlado de contexto, somadas a uma visão geral da transcrição.

Exportação: transcrição em TXT e PDF; resumo estruturado em PDF e JSON.



Decisões de projeto, reprodutibilidade e verificação

- Arquitetura em camadas — o núcleo de IA (src/) é independente das camadas de uso (backend/): linha de comando, API FastAPI, painel Streamlit e front-end Next.js.
 - Consequência metodológica: o núcleo pode ser testado e reutilizado sem subir interface.
- Configuração externalizada — modelo de ASR, modelo de embeddings, sumarizador, tamanho de janela, sobreposição, limiar de similaridade e limites de upload são variáveis de ambiente.
 - Consequência metodológica: um experimento é descrito por um conjunto de variáveis, não por uma alteração de código — o que torna a rodada reprodutível.
- Instrumentação — toda execução devolve stage_timings: tempo de pré-processamento, transcrição, análise semântica, sumarização, exportação e tempo total de parede.
- Verificação — suíte automatizada em pytest com dublês de modelos: 15 testes em 6 módulos cobrindo transcrição, embeddings, sumarização, validação de upload, API e integração.
- Protocolo experimental — 2 a 4 cenários de áudio com perfis distintos: curto e limpo; médio com múltiplos falantes; com ruído de fundo; com silêncio inicial prolongado.
 - Para cada cenário registram-se configuração, métricas automáticas e checklist qualitativo.

RESULTADOS E DISCUSSÃO

Entrega funcional: o que o sistema efetivamente faz

- Pipeline integrado e operacional de ponta a ponta, acessível por três vias: linha de comando, API REST (GET /health, POST /transcribe) e painel Streamlit, além de front-end web.
- Saídas produzidas por execução:
 - transcrição com marcação temporal por segmento no formato [HH:MM:SS --> HH:MM:SS];
 - blocos semânticos rotulados, com intervalo de tempo aproximado na linha do tempo da reunião;
 - resumo em três seções — decisões, pendências e próximos passos — mais uma visão geral;
 - exportação em TXT, PDF e JSON, permitindo consumo humano e consumo por outro sistema.
- Robustez operacional: validação de upload (limite de 50 MiB e extensões .wav/.mp3/.m4a/.webm/.ogg), redução de ruído em blocos para gravações longas e empacotamento em Docker.
- Ferramentas de medição entregues: `benchmark_pipeline.py` agrega métricas de vários áudios em tabela; `compare_whisper_models.py` executa comparação A/B entre tamanhos de modelo.

Discussão: a modularidade foi validada na prática — o agrupamento foi reimplementado em NumPy puro, eliminando a dependência de FAISS, sem qualquer alteração nas camadas de interface.

RESULTADOS E DISCUSSÃO

Avaliação experimental, limitações e ameaças à validade

- Métricas instrumentadas por execução: duração do áudio, tempo de cada etapa, tempo total, fator de tempo real ($RTF = \text{tempo total} \div \text{duração do áudio}$), número de segmentos, número de blocos semânticos e extensão da transcrição; WER/CER quando há transcrição de referência.

Cenário de áudio	Duração (s)	Tempo total (s)	RTF	Segmentos	Blocos
A1 — curto, um locutor, ambiente calmo	—	—	—	—	—
A2 — médio, múltiplos falantes	—	—	—	—	—
A3 — com ruído de fundo / microfone distante	—	—	—	—	—

Tabela 1 — Rodada de referência (scripts/benchmark_pipeline.py; configuração conforme docs/evaluation_protocol.md).

Limitações e ameaças à validade

- Idioma fixado em português: o sistema não trata reuniões multilíngues.
- Agrupamento guloso por limiar — a ordem de chegada influencia os blocos, pois não há otimização global como em agrupamento hierárquico ou espectral.
- O sumariizador (T5 pequeno, treinado majoritariamente em inglês) é o gargalo de qualidade: a transcrição é consistentemente melhor que o resumo.
- Ausência de diarização (o sistema registra o que foi dito, não quem disse) e avaliação qualitativa conduzida por um único avaliador sobre um conjunto reduzido de áudios.

CONSIDERAÇÕES FINAIS

Conclusão

O objetivo foi atingido no nível de sistema: existe um pipeline reproduzível que converte áudio de reunião em português em documentação estruturada e rastreável ao tempo do áudio, com instrumentação de desempenho em todas as etapas.

Contribuição

- Uma arquitetura de referência aberta e modular que integra ASR, estruturação semântica e sumarização, executável localmente — sem enviar o áudio da reunião a terceiros.
- Um protocolo de avaliação e ferramentas de medição que permitem comparar configurações de forma reproduzível, e não apenas relatar uma execução isolada.

Aprendizados

- A separação entre núcleo de IA e camadas de uso foi decisiva para evoluir o sistema sem retrabalho.
- Medir por etapa, e não apenas o tempo total, foi o que revelou onde o custo realmente está.

Trabalhos futuros

- sumarizador instruído em português (mT5/PTT5 ou modelo de linguagem com prompt estruturado);
- diarização de falantes, para atribuir decisões a seus responsáveis;
- agrupamento com otimização global, substituindo a estratégia gulosa por limiar;
- avaliação com WER/CER sobre corpus anotado e julgamento humano do resumo por múltiplos avaliadores.

REFERÊNCIAS

- ALEMI, A. A.; GINSPARG, P. Text segmentation based on semantic word embeddings. arXiv:1503.05543, 2015.
- McFEE, B. et al. librosa: audio and music signal analysis in Python. In: PROCEEDINGS OF THE 14th PYTHON IN SCIENCE CONFERENCE (SciPy), 2015. p. 18-25.
- RADFORD, A. et al. Robust speech recognition via large-scale weak supervision. arXiv:2212.04356, 2022.
- RAFFEL, C. et al. Exploring the limits of transfer learning with a unified text-to-text transformer. Journal of Machine Learning Research, v. 21, n. 140, p. 1-67, 2020.
- REIMERS, N.; GUREVYCH, I. Sentence-BERT: sentence embeddings using Siamese BERT-networks. In: PROCEEDINGS OF THE 2019 CONFERENCE ON EMPIRICAL METHODS IN NATURAL LANGUAGE PROCESSING (EMNLP-IJCNLP), 2019. p. 3982-3992.

Conferir a formatação final conforme a NBR 6023 adotada pela unidade acadêmica.

AGRADECIMENTOS



À Universidade Presbiteriana Mackenzie e ao Programa de Iniciação Científica, pelo apoio institucional à pesquisa.
Ao Prof. Victor Inácio de Oliveira, pela orientação ao longo do projeto.